

HỌC PHẦN NÂNG CAO VỚI C++



MỤC LỤC

+ - * / %
& | ^ ~ < < > >

Chương 3

- 3.1. Các nguyên tắc của nạp chồng toán tử
- 3.2. Các giới hạn
- 3.3. Nạp chồng các toán tử 2 ngôi
- 3.4. Nạp chồng các toán tử 1 ngôi
- 3.5. Nạp chồng các toán tử đặc biệt [] và ()
- 3.6. Nạp chồng toán tử chuyển đổi kiểu
- 3.7. Nạp chồng toán tử New và Delete
- 3.8. Nạp chồng toán tử chèn dòng và trích dòng
- 3.9. Các ví dụ và bài tập chương 3



Mục tiêu bài học

- ❖ - Hiểu được **khái niệm** và **nguyên tắc** nạp chồng toán tử.
- ❖ - Biết được **giới hạn** của nạp chồng.
- ❖ - Thực hiện được nạp chồng cho **toán tử một ngôi** và **hai ngôi**.
- ❖ - Nắm được cách nạp chồng các toán tử đặc biệt như **[]**, **()**, **<<**, **>>**, **new**, **delete**.
- ❖ - Ứng dụng nạp chồng toán tử trong thiết kế class.

C++ is a language for defining and using powerful abstractions



Q: Tại sao phải nạp chồng toán tử

❖ Giả sử chúng ta có class A và thử nghiệm cộng 2 đối tượng bằng toán tử '+'

```
class A {  
    // class definition  
};
```

```
A a1, a2, a3;
```

```
a3 = a1 + a2; // Error! '+' not defined for class A
```

=> C++ không nhận diện được phép '+' cho 2 đối tượng mà người dùng định nghĩa



Khái niệm:

- ❖ Nạp chồng toán tử (Operator Overloading) cho phép định nghĩa lại cách hoạt động của một toán tử C++ khi áp dụng cho đối tượng của lớp do người dùng định nghĩa.

Mục tiêu:

- ❖ Chỉ có thể nạp chồng các toán tử có sẵn trong C++.
- ❖ Không thể tạo toán tử mới.
- ❖ Toán tử giữ nguyên **ưu tiên** và **thứ tự kết hợp**.
- ❖ Ít nhất một toán hạng phải là **kiểu do người dùng định nghĩa** (class/struct).
- ❖ Có thể nạp chồng dưới dạng **hàm thành viên** hoặc **hàm bạn**.



Các nguyên tắc cơ bản:

- ❖ **Chỉ nạp chồng được toán tử có sẵn** trong C++ (không thể tạo toán tử mới).
- ❖ **Ít nhất một toán hạng** phải là kiểu do người dùng định nghĩa (class/struct).
- ❖ **Không thay đổi ưu tiên (precedence) và tính kết hợp (associativity)** của toán tử.
- ❖ Có thể định nghĩa dưới dạng:
 - **Hàm thành viên** (member function).
 - **Hàm bạn** (friend function).
- ❖ Nạp chồng **không thay đổi số lượng toán hạng** của toán tử.
- ❖ Giữ nguyên ý nghĩa ngữ pháp để tránh gây nhầm lẫn.



Nạp chồng toán tử '+'

```
1  ✓ class Complex {  
2      double real, imag;  
3  public:  
4      Complex(double r=0, double i=0) : real(r), imag(i) {}  
5  ✓      Complex operator+(const Complex &c) {  
6          return Complex(real + c.real, imag + c.imag);  
7      }  
8  };  
9
```



Các toán tử không thể nạp chồng

- ❖ :: (scope resolution)
- ❖ . (member access)
- ❖ .* (pointer-to-member access)
- ❖ sizeof
- ❖ typeid
- ❖ ?: (ternary)

Why?

- ❖ Đây là **toán tử mức ngôn ngữ** được xử lý tại compile time, nên C++ không cho phép lập trình viên thay đổi.



Toán tử 2 ngôi (Binary Operators) là gì?:

❖ Toán tử 2 ngôi là các toán tử làm việc với 2 toán hạng.

Cách cài đặt:

Cách	Đặc điểm	Khi dùng
Member function	Nhận 1 đối số (toán hạng trái là <code>this</code>)	Khi toán hạng trái luôn là object của class
Non-member friend function	Nhận 2 đối số	Khi toán hạng trái không phải object của class

Các toán tử phổ biến

❖ `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `<=`, `>`, `>=`, `&&`, `||`.



Dạng member-function:

```
1 ✓ class Vector2D {  
2     double x, y;  
3     public:  
4         Vector2D(double x=0, double y=0) : x(x), y(y) {}  
5 ✓     Vector2D operator+(const Vector2D &o) const { // Member  
6         return Vector2D(x + o.x, y + o.y);  
7     }  
8     double length() const { return sqrt(x*x + y*y); }  
9     void display() const { cout << "(" << x << ", " << y << ")"; }  
10 };  
11
```



Dạng non-member friend function:

```
1  class Vector2D {  
2      double x, y;  
3  public:  
4      Vector2D(double x=0, double y=0) : x(x), y(y) {}  
5      friend Vector2D operator*(double scalar, const Vector2D &v); // Friend  
6  };  
7  
8  Vector2D operator*(double scalar, const Vector2D &v) {  
9      return Vector2D(v.x * scalar, v.y * scalar);  
10 }  
11
```

Note: Với dạng friend function cần phải khai báo với từ khóa "**friend**" bên trong class để có thể thực hiện được việc gọi hàm

VÍ DỤ GỌI TRONG MAIN



```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4
5  class Vector2D {
6      double x, y;
7  public:
8      Vector2D(double x=0, double y=0) : x(x), y(y) {}
9      Vector2D operator+(const Vector2D &o) const { return Vector2D(x + o.x, y + o.y); } // Member
10     friend Vector2D operator*(double scalar, const Vector2D &v); // Friend
11     void display() const { cout << "(" << x << ", " << y << ")"; }
12 };
13
14 Vector2D operator*(double scalar, const Vector2D &v) {
15     return Vector2D(v.x * scalar, v.y * scalar);
16 }
17
18 int main() {
19     Vector2D v1(3,4), v2(1,2);
20     Vector2D sum = v1 + v2; // Member operator+
21     cout << "v1 + v2 = "; sum.display(); cout << endl;
22
23     Vector2D scaled = 2.5 * v1; // Friend operator*
24     cout << "2.5 * v1 = "; scaled.display(); cout << endl;
25 }
26
```



Toán tử 1 ngôi (Unary Operators) là gì?:

❖ Toán tử 1 ngôi là các toán tử tác động lên **một toán hạng** duy nhất.

Cách cài đặt:

Cách	Số đối số cần truyền khi overload
Member function	0 đối số (toán hạng là <code>this</code>)
Friend function	1 đối số (object cần áp dụng toán tử)

Các toán tử phổ biến

❖ `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `<=`, `>`, `>=`, `&&`, `||`.



Dạng member-function:

```
1 ✓ class Vector2D {  
2     double x, y;  
3     public:  
4         Vector2D(double x=0, double y=0) : x(x), y(y) {}  
5 ✓     Vector2D operator+(const Vector2D &o) const { // Member  
6         return Vector2D(x + o.x, y + o.y);  
7     }  
8     double length() const { return sqrt(x*x + y*y); }  
9     void display() const { cout << "(" << x << ", " << y << ")"; }  
10 };  
11
```



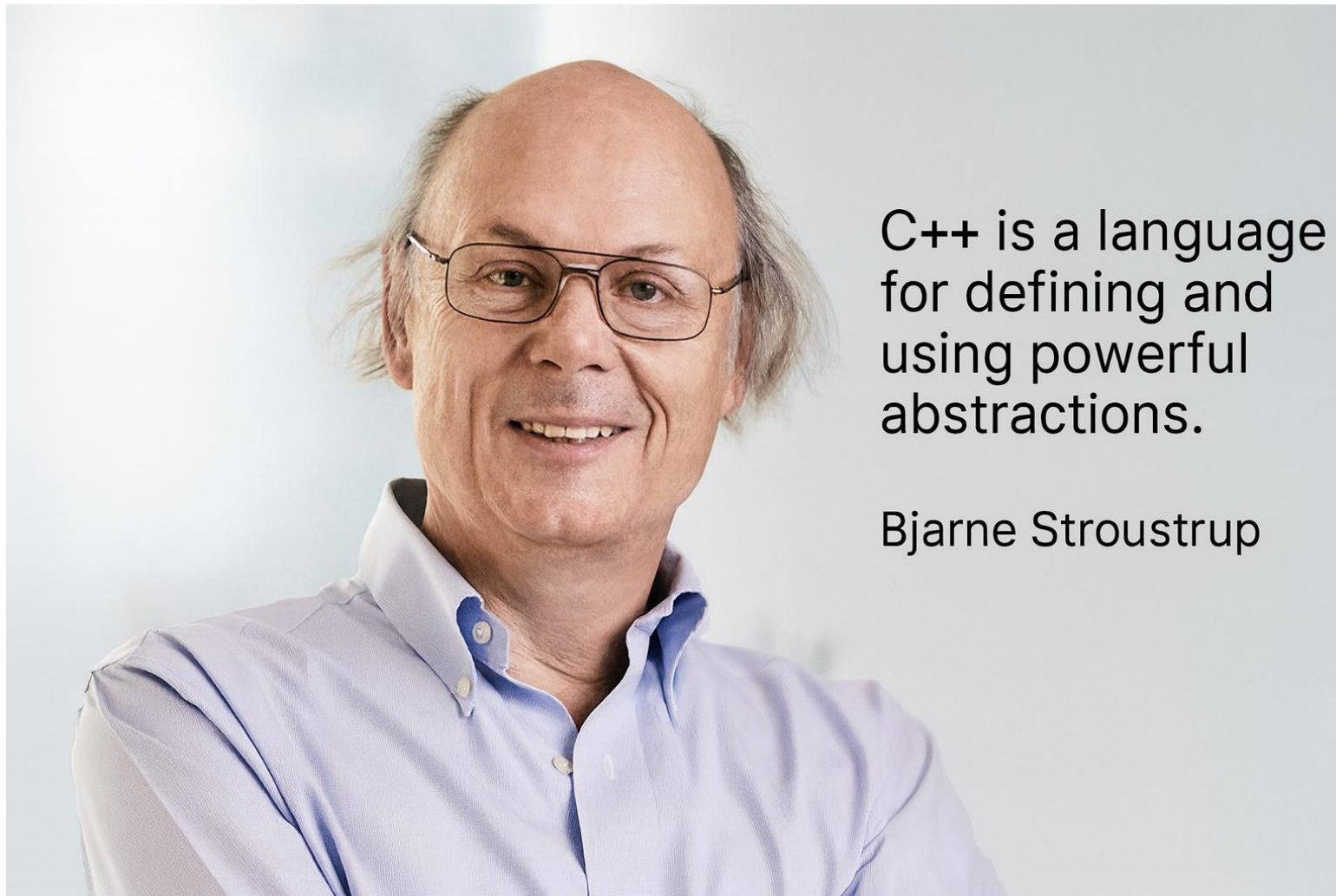
Dạng non-member friend function:

```
1 ✓ class Vector2D {
2     double x, y;
3     public:
4         Vector2D(double x=0, double y=0) : x(x), y(y) {}
5         friend Vector2D& operator++(Vector2D &v); // Prefix ++
6         void display() const { cout << "(" << x << ", " << y << ")"; }
7     };
8
9     // Định nghĩa ngoài class
10 ✓ Vector2D& operator++(Vector2D &v) {
11     v.x += 1; v.y += 1;
12     return v;
13 }
14
15 ✓ int main() {
16     Vector2D v(1, 2);
17     ++v; // Gọi friend operator++
18     v.display(); // Output: (2, 3)
19 }
--
```

VÍ DỤ GỌI TRONG MAIN



```
1  class Vector2D {
2      double x, y;
3  public:
4      Vector2D(double x=0, double y=0) : x(x), y(y) {}
5
6      // Nạp chồng toán tử 1 ngôi (-)
7  Vector2D operator-() const {
8      return Vector2D(-x, -y);
9  }
10
11     void display() const { cout << "(" << x << ", " << y << ")"; }
12 };
13
14 int main() {
15     Vector2D v(3, -4);
16     Vector2D neg = -v; // Gọi member operator-
17     neg.display(); // Output: (-3, 4)
18 }
19
```

C++ is a language
for defining and
using powerful
abstractions.

Bjarne Stroustrup



Toán tử [] Cho phép đối tượng được truy cập như mảng

```
1 ✓ double& operator[](int index) {  
2     if (index < 0 || index > 1) throw out_of_range("Index out of range");  
3     return (index == 0) ? x : y;  
4 }  
5 ✓ const double& operator[](int index) const {  
6     if (index < 0 || index > 1) throw out_of_range("Index out of range");  
7     return (index == 0) ? x : y;  
8 }  
9
```

Toán tử () Cho phép đối tượng hoạt động như một hàm (functor).

```
1 // Thiết lập lại giá trị x, y  
2 ✓ void operator()(double newX, double newY) {  
3     x = newX; y = newY;  
4 }  
5
```



Toán tử chuyển đổi kiểu Cho phép đối tượng được chuyển đổi sang kiểu dữ liệu khác, hỗ trợ cả chuyển đổi tường minh và ngầm định.

```
1  ✓ operator double() const {  
2      return sqrt(x*x + y*y); // độ dài vector  
3  }  
4  ✓ explicit operator string() const {  
5      return "(" + to_string(x) + ", " + to_string(y) + ")";  
6  }  
7  ✓ operator bool() const {  
8      return (x != 0 || y != 0);  
9  }  
10
```



Toán tử new/delete Giúp tùy biến cách cấp phát và giải phóng bộ nhớ khi tạo hoặc xóa đối tượng.

```
1  void* operator new[](size_t size) {  
2      cout << "Allocating array of " << size << " bytes\n";  
3      void* ptr = malloc(size);  
4      if (!ptr) throw bad_alloc();  
5      return ptr;  
6  }  
7  void operator delete[](void* ptr) {  
8      cout << "Freeing array memory\n";  
9      free(ptr);  
10 }  
11
```



Toán tử “<<” và “>>” Cho phép định nghĩa cách nhập và xuất dữ liệu của đối tượng qua luồng.

```
1  friend ostream& operator<<(ostream &out, const Vector2D &v) {  
2      out << fixed << setprecision(2) << "(" << v.x << ", " << v.y << " )";  
3      return out;  
4  }  
5  friend istream& operator>>(istream &in, Vector2D &v) {  
6  if (!(in >> v.x >> v.y)) {  
7      throw invalid_argument("Invalid input for Vector2D");  
8  }  
9      return in;  
10 }  
11
```



Main function:

```
1  ✓ int main() {  
2      Vector2D v1(3, 4), v2(1, 2);  
3  
4      cout << "v1 = " << v1 << ", v2 = " << v2 << endl;  
5      cout << "v1 + v2 = " << v1 + v2 << endl;  
6      cout << "v1 - v2 = " << v1 - v2 << endl;  
7      cout << "-v1 = " << -v1 << endl;  
8  
9      ++v1; cout << "++v1 = " << v1 << endl;  
10  
11     cout << "v1[0] = " << v1[0] << ", v1[1] = " << v1[1] << endl;  
12     cout << "Dot product = " << v1(v2) << endl;  
13  
14     v1(5, 6); cout << "After reset v1 = " << v1 << endl;  
15  
16     double length = v1;  
17     cout << "Length of v1 = " << length << endl;  
18  
19     string s = static_cast<string>(v1);  
20     cout << "String form = " << s << endl;  
21  
22     Vector2D* arr = new Vector2D[2];  
23     delete[] arr;  
24 }  
25
```

Out put:

```
v1 = (3.00, 4.00), v2 = (1.00, 2.00)  
v1 + v2 = (4.00, 6.00)  
v1 - v2 = (2.00, 2.00)  
-v1 = (-3.00, -4.00)  
++v1 = (4.00, 5.00)  
v1[0] = 4.00, v1[1] = 5.00  
Dot product = 14.00  
After reset v1 = (5.00, 6.00)  
Length of v1 = 7.81  
String form = (5.000000, 6.000000)  
Allocating array of <implementation-defined> bytes  
Freeing array memory
```



- ❖ *In C++, everything is possible... and that's the problem.*
- ❖ *C++: Where your friends have access to your private parts.*
- ❖ *There are only two kinds of programming languages: those people complain about, and those nobody uses. C++ is ...*



Viết lớp Vector2D biểu diễn véc-tơ 2 chiều (x, y) và nạp chồng các toán tử sau:

1. Toán tử hai ngôi: +, -, ==, !=

- So sánh bằng dùng sai số epsilon = 1e-9.

2. Toán tử một ngôi: - (đổi dấu), ++ (prefix & postfix).

3. Toán tử đặc biệt:

- operator[](int) (non-const và const).
- Chỉ số 0→x, 1→y, sai ném std::out_of_range.
- operator()(const Vector2D&) const tính **dot product**.
- operator()(double, double) đặt lại (x, y).
- operator double() const trả về **độ dài**.
- explicit operator std::string() const trả về dạng chuỗi "(x,y)".

4. I/O: operator<<, operator>>

- << in dạng (x, y) với 2 chữ số thập phân.
- >> đọc 2 số thực; nhập lỗi ném std::invalid_argument.

5. Quản lý bộ nhớ tùy biến: operator new/delete và operator new[]/delete[] (in log khi cấp phát/giải phóng, kiểm tra null → ném std::bad_alloc).

6. Viết main() minh họa đầy đủ các toán tử trên.

Yêu cầu: dùng const-correctness, tham số truyền bằng const& khi phù hợp; giải thích ngắn vì sao explicit cần thiết (1–2 câu trong comment).

THANK YOU



Học viện Ngân hàng

12 P. Chùa Bộc, P. Kim Liên, TP. Hà Nội

T: 1900 561 595 **F:** 1900 561 595

E: truyenthong@hvnh.edu.vn

W: www.hvnh.edu.vn